

web1

2. 主观题 (9分)

2.

编写一个Servlet，获得用户信息列表，然后跳转到user_list.jsp页面，在此页面中使用JSTL标签展示用户列表信息。假设userService对象有方法listAll()返回用户列表List<User>，用户信息：ID(id)、姓名(name)、密码(passwd)。

- 1)、编写Servlet类的doPost(HttpServletRequest, HttpServletResponse)方法；
- 2)、在web.xml文件中注册此Servlet类；(类名：com.web.UserListServlet)；
- 3)、使用JSTL标签编写展示用户列表JSP页面中的table部分。

(1)

```
private void doPost(HttpServletRequest req, HttpServletResponse res) throws
Exception{
    List<User> list = new userService.listAll();
    req.setAttribute("list", list);
    res.getRequestDispatcher("/user_list.jsp").forward(req, res);
}
```

(2)

```
<servlet>
    <servlet-name>UserListServlet</servlet-name>
    <servlet-class>com.web.UserListServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>UserListServlet</servlet-name>
    <url-pattern>/userList</url-pattern>
</servlet-mapping>
```

(3)

```
<table>
    <tr>
        <th>序号</th>
        <th>ID</th>
        <th>姓名</th>
        <th>密码</th>
    </tr>
    <c:forEach items="${list}" var="s" varStatus="idx">
        <tr>
            <td>${idx.index+1}</td>
            <td>${s.id}</td>
            <td>${s.name}</td>
            <td>${s.password}</td>
        </tr>
    </c:forEach>
</table>
```

29、编写一个用户注册的JSP页面，效果图如下：

3.

1)、使用html编写出上图效果的页面。

2)、提交表单时使用JS验证：①账号为4-8个字符；②密码不能为空，且二次密码必须相同。（可使用jQuery获得元素）

```
(1)
<!DOCTYPE html>
<html>
  <head>
    <title>用户注册</title>
  </head>
  <body>
    <center>
      <h1>用户注册</h1><hr>
      <form action="#" method="post" onsubmit="return checkval()">
        <table>
          <tr>
            <td align="right">账号: </td>
            <td><input type="text" name="name" id="name"></td>
          </tr>
          <tr>
            <td align="right">密码: </td>
            <td><input type="password" name="passwd" id="passwd">
          </td>
          </tr>
          <tr>
            <td align="right">确认密码: </td>
            <td><input type="password" name="passwd2" id="passwd2">
          </td>
          </tr>
          <tr>
            <td colspan="2" align="center">
              <input type="submit" value="确定">
              <input type="reset" value="取消">
            </td>
          </tr>
        </table>
      </form>
    </center>
  </body>
</html>
```

(2)

```
<script>
    function checkval() {
        var nameElm = document.getElementById('name');
        var passwdElm = document.getElementById('passwd');
        var passwd2Elm = document.getElementById('passwd2');
        if(!/^\\w{4,8}$/.test(nameElm.value)) {
            alert('账号必须为[4,8]位字符');
            nameElm.focus();
            return false;
        }
        if(passwdElm.value == '') {
            alert('密码不能为空');
            passwdElm.focus();
            return false;
        }
        if(passwdElm.value != passwd2Elm.value) {
            alert('两次密码不相同');
            passwd2Elm.value = '';
            passwd2Elm.focus();
            return false;
        }
        return true;
    }
</script>
```

4. 主观题 (9分)

4. 假设有一个过滤器，其全限定名为：`com.web.EncodingFilter`，要求它能拦截所有的请求，且可传入字符集参数`charset`。请在`web.xml`文件中注册此过滤器。

```
<filter>
    <filter-name>EncodingFilter</filter-name>
    <filter-class>com.web.EncodingFilter</filter-class>
    <init-param>
        <param-name>charset</param-name>
        <param-value>UTF-8</param-value>
    </init-param>
</filter>
<filter-mapping>
    <filter-name>EncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

5. 主观题 (9分)

5.

请详细描述Servlet的生命周期。

servlet有良好的生存期的定义，包括加载和实例化、初始化、处理请求以及服务结束。这个生存期由javax.servlet.Servlet接口的init,service和destroy方法表达。描述Servlet请求的流程。请HTTP协议基于请求响应模式，客户端向服务器发送一个请求，服务器则以一个状态行为作为响应。

- (1) 第1次访问servlet时，实例化Servlet类并调用init方法进行初始化。
- (2) 每次访问servlet时，调用service方法，service方法根据请求的类型，再调用相应的doGet或doPost()方法。
- (3) 当Servlet容器关闭或重启时，调用destroy方法销毁servlet对象。
- (4) Servlet默认是单实例的。

6.主观题 (9分)

6. 在JavaScript中，编写一个用户(User)类，属性有名字 (name)、年龄 (age)，方法有show()，在方法中使用alert弹出用户的名字和年龄。定义一个对象并调用方法。

```
<script>
function User(name, age){
    this.name = name;
    this.age = age;
}
User.prototype.show=function(){
    alert(this.name + ',' + this.age);
}
var user = new User("test", 20);
user.show();
</script>
```

7.主观题 (9分)

7. 在JavaScript中，对数组[6, 3, 1, 5, 2, 4]按数值进行从大到小排序，并将结果以分号相隔拼成一个字符串，并使用alert弹出该字符串。

```
<script>
var arr = [6,3,1,5,2,4];
arr.sort(function(a, b){
    return b - a;    //从大到小排序
});
var str = arr.join(';');
alert(str);
</script>
```

web2

2.主观题 (9分)

编写一个Servlet 获得部门信息列表，然后跳转到dept_list.jsp 页面，在此页面中使用JSTL 标签展示部门列表信息。已定义部门实体类Dept，其属性信息：ID(id)、部门名称(name)、部门主管(master)、部门人数(cnt)。

2. (1)、实现DeptService 类的listAll()方法，返回所有部门列表List<Dept>。(建议使用dbutils，假设已获得Connection 类型对象conn);
- (2)、编写Servlet 类的doPost(HttpServletRequest, HttpServletResponse)方法，并使用注解方式或在web.xml 中注册该Servlet;
- (3)、使用JSTL 标签编写展示部门信息列表页面中的完整table 部分，要展示序号、部门名称、部门主管、部门人数信息。

```
(1)
public class DeptService {
    public List<Dept> listAll(){
        QueryRunner runner = new QueryRunner();
        String sql = "select * from tb_dept";
        List<Dept> list = runner.query(conn, sql, new BeanListHandler<
        (Dept.class));
        return list;
    }
}
```

```
(2)
@WebServlet("/dept/list")
public class DeptListServlet extends HttpServlet {
    private DeptService service = new DeptService();
    public void doPost(HttpServletRequest req, HttpServletResponse resp) throws
    Exception {
        List<Dept> list = service.listAll();
        req.getSession().setAttribute("list", list);
        req.getRequestDispatcher("/dept_list.jsp").forward(req, resp);
    }
}
```

```
(3)
<table>
    <tr>
        <td>序号</td>
        <td>部门名称</td>
        <td>部门主管</td>
        <td>部门人数</td>
    </tr>
    <c:forEach var="s" items="${list}" varStatus="idx">
        <tr>
            <td>${idx.count}</td>
            <td>${s.name}</td>
            <td>${s.master}</td>
            <td>${s.cnt}</td>
        </tr>
    </c:forEach>
</table>
```

3. 主观题 (9分)

编写一个增加部门功能的页面，效果图如下：

增加部门

3.

部门名称：

部门主管：

部门人数：

(1)、使用html 编写出上图效果的页面。

(2)、点击“确定”按钮时，使用Javascript 验证部门名称、主管、人数不能为空，部门名称为4-8 个大小写字母组成，部门人数的数值范围为10-100 的整数。

```
<!DOCTYPE html>
<html>
<head>
<title>增加班级</title>
<script>
function checkval() {
    var nameElm = document.getElementById('name');
    var cntElm = document.getElementById('cnt');
    var masterElm = document.getElementById('master');
    if (nameElm.value == '') {
        alert('部门名称不能为空');
        nameElm.focus();
        return false;
    }
    if (/^[a-zA-Z]{4,8}$/.test(nameElm.value)) {
        alert('部门名称应为4-8个字母');
        nameElm.focus();
        return false;
    }
    if (masterElm.value == '') {
        alert('部门主管不能为空');
        masterElm.focus();
        return false;
    }
    if (cntElm.value == '') {
        alert('部门人数不能为空');
        cntElm.focus();
        return false;
    }
    if (parseInt(cntElm.value) >= 10
        && parseInt(cntElm.value) <= 100) {
        alert('人数必须为[10,100]范围内整数');
        cntElm.focus();
        return false;
    }
    return true;
}
</script>
</head>
<body>
<center>
```

```

<h1>增加部门</h1>
<hr>
<form action="#" method="post" onsubmit="return checkval()">
  <table>
    <tr>
      <td>部门名称: </td>
      <td>
        <input type="text" name="name" id="name">
      </td>
    </tr>
    <tr>
      <td align="right">部门主管: </td>
      <td>
        <input type="text" name="master" id="master">
      </td>
    </tr>
    <tr>
      <td align="right">部门人数: </td>
      <td>
        <input type="text" name="cnt" id="cnt">
      </td>
    </tr>
    <tr>
      <td colspan="2">
        <input type="submit" value="确定">
        <input type="reset" value="取消">
      </td>
    </tr>
  </table>
</form>
</center>
</body>
</html>

```

4. 主观题 (9分)

请简述JSP 中的九种内置对象及作用。

- (1)request对象: 表示HTTP请求, 提供对请求参数和属性的访问。
- (2)response对象: 表示HTTP响应, 提供对响应状态和头信息的访问。
- (3)pageContext对象: 提供对JSP页面上下文的访问, 包括请求、响应、会话和应用程序对象。
- (4)session对象: 表示HTTP会话, 提供对会话属性的访问。
- (5)out对象: 用于向响应输出流写入内容。
- (6)config对象: 提供对JSP页面配置信息的访问, 包括JSP引擎和上下文参数。
- (7)page对象: 提供对当前JSP页面的访问。
- (8)exception对象: 在JSP页面中出现异常时, 该对象提供对异常信息的访问。
- (9)application对象: 可将信息保存在服务器中, 直到服务器关闭, 否则application对象中保存的信息会整个应用中都有。

5.主观题 (9分)

5.

请简要描述MVC 设计模式及其应用。

MVC是一种常用的软件设计模式，它将应用程序分为模型（Model）、视图（View）和控制器（Controller）三个部分，以实现应用程序的松耦合、高内聚和可扩展性。在MVC模式中，模型（Model）表示应用程序的数据和业务逻辑，视图（View）表示用户界面，控制器（Controller）协调模型和视图之间的交互，将用户的操作转化为对模型的操作，并将模型的状态更新到视图中。MVC模式的应用可以使代码更易于维护和扩展，因为不同的功能被分配到不同的模块中，开发人员可以更容易地修改和测试每个模块。此外，MVC模式也有助于实现代码的可重用性，因为模型和视图可以在不同的应用程序中重复使用。

6.主观题 (9分)

6.

请简要介绍一下JavaScript 中的异步编程。

JavaScript中的异步编程是指在代码执行过程中，不需要等待某些操作完成就可以继续执行后续代码的一种编程方式。在JavaScript中，常见的异步编程方式包括回调函数、Promise和async/await。回调函数是一种最基础的异步编程方式，它通过将需要异步执行的代码封装在一个函数中，并在操作完成后调用该函数来实现异步执行。例如，可以在setTimeout()函数中传递一个回调函数，在延迟指定时间后执行该回调函数。Promise是一种更高级的异步编程方式，它可以更好地处理异步操作的结果和错误。Promise可以将异步操作封装成一个Promise对象，并使用.then()和.catch()方法处理异步操作的结果和错误。async/await是ES2017中引入的一种异步编程方式，它可以更直观地编写异步代码。使用async/await可以将异步操作看作同步操作，通过async函数定义异步函数，使用await关键字等待异步操作完成并返回结果。异步编程可以提高代码的性能和可维护性，避免了阻塞线程等问题，因此在JavaScript中被广泛应用。

7.主观题 (9分)

7.

请简要介绍一下setTimeout()和setInterval()定时器函数。

setTimeout()函数接受两个参数：要执行的函数和延迟的毫秒数。它会在延迟指定的时间后执行一次函数。setInterval()函数也接受两个参数：要执行的函数和时间间隔的毫秒数。它会每隔指定的时间间隔执行一次函数，直到调用clearInterval()函数停止执行。setTimeout()和setInterval()函数返回一个定时器ID，可以用来取消定时器。可以通过调用clearTimeout()函数或clearInterval()函数来取消定时器。